

LAPORAN PRAKTIKUM PEMROGRAMAN WEB

Modul 11: Pengenalan React



Oleh:

Zalfaa Ghaitsaa Arrahma Bahtiar

707012500077

D4SIKC-49-04

TELKOM UNIVERSITY

2026

A. TUJUAN PRAKTIKUM

Setelah menyelesaikan modul ini, peserta diharapkan mampu:

1. Memahami konsep dasar React sebagai library JavaScript untuk membangun antarmuka pengguna (UI) berbasis component dan declarative programming.
2. Memahami arsitektur UI berbasis component serta perbedaan pendekatan React dibandingkan manipulasi DOM tradisional (Vanilla JS).
3. Melakukan setup environment React dan membuat aplikasi React menggunakan tool modern (Vite / Create React App).
4. Membuat dan mengelola component menggunakan JSX serta memahami struktur file aplikasi React.
5. Menggunakan props untuk mengirim data antar component secara satu arah (one-way data flow).
6. Mengelola state untuk membangun interaktivitas dasar pada aplikasi React.
7. Melakukan rendering list dan conditional rendering untuk menampilkan data secara dinamis.
8. Membangun aplikasi React sederhana yang menerapkan component, props, state, dan rendering dinamis secara terstruktur.

B. ALAT DAN BAHAN

Untuk menjalankan modul ini, diperlukan:

1. Node.js (versi LTS) -> Runtime JavaScript untuk menjalankan React dan dependency manager (npm).
2. Package Manager (npm atau yarn) -> Digunakan untuk mengelola dependensi proyek React.
3. React Library -> Digunakan untuk membangun antarmuka pengguna berbasis component.
4. Build Tool (Vite atau Create React App) -> Untuk membuat dan menjalankan proyek React dengan environment modern.
5. Text Editor / IDE -> Visual Studio Code (direkomendasikan) dengan ekstensi pendukung JavaScript & React.
6. Web Browser -> Google Chrome atau Mozilla Firefox untuk menjalankan dan debugging aplikasi React.
7. Pemahaman Dasar HTML, CSS, dan JavaScript (ES6) -> Sebagai prasyarat sebelum mengikuti praktikum React.
8. Telah menyelesaikan praktikum JavaScript pada modul sebelumnya -> Termasuk pemahaman arrow function, array method (map, filter), dan module system.

C. LANGKAH LANGKAH PRAKTIKUM

1) Studi Kasus 1

- Menginstall Node.js
- Membuat folder yang ingin digunakan dengan membuka terminal dengan perintah
mkdir react-app
cd react-app
- Lalu lanjutkan dengan mengikuti step step berikut

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\xampp\htdocs\Pemograman_Web\WebPro4904>mkdir react-app
C:\xampp\htdocs\Pemograman_Web\WebPro4904>cd react-app
C:\xampp\htdocs\Pemograman_Web\WebPro4904\react-app>npm create vite@latest
> npx
> create-vite

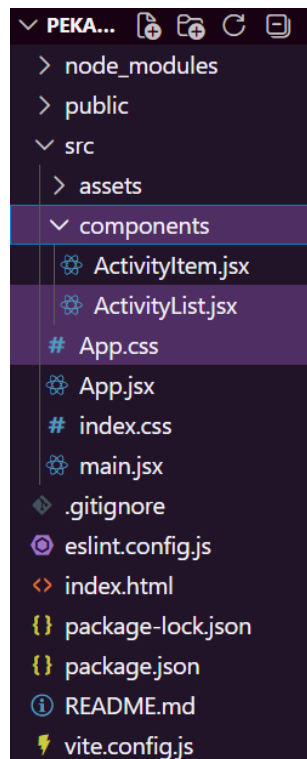
|
| o Project name:
|   Pekan-11
|
| o Package name:
|   pekan-11
|
| o Select a framework:
|   React
|
| o Select a variant:
|   JavaScript
|
| o Install with npm and start now?
|   Yes
|
| o Scaffolding project in C:\xampp\htdocs\Pemograman_Web\WebPro4904\react-app\Pekan-11...
|
|
| o Installing dependencies with npm...
|
added 135 packages, and audited 136 packages in 56s
31 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
|
| o Starting dev server...
|
> pekan-11@0.0.0 dev
> vite

Port 5173 is in use, trying another one...

VITE v8.0.14 ready in 2962 ms
  → Local:   http://localhost:5174/
  → Network: use --host to expose
  → press h + enter to show help
Terminate batch job (Y/N)? y
C:\xampp\htdocs\Pemograman_Web\WebPro4904\react-app>
```

Gambar C.1.1 Alur pembuatan project

d) Membuat folder dengan format atau penempatan seperti ini:



Gambar C.1.2 Format Folder

e) Membuat codingan nya di VS Code sesuai dengan tugas praktikum

```
src > App.jsx > App
1 // Parent Component utama aplikasi
2 // useState adalah React Hook untuk menyimpan data yang bisa berubah
3 // Setiap kali state berubah, React otomatis me-render ulang tampilan
4 import { useState } from "react";
5
6 import ActivityList from "../components/ActivityList";
7 import "../App.css";
8
9 function App() {
10 // useState(initialValue) untuk mengembalikan [nilaiState, fungsiUbahState]
11 // 'activities' = array data aktivitas yang tampil di layar
12 // 'setActivities' = fungsi untuk mengubah data activities
13 const [activities, setActivities] = useState([
14   { id: 1, text: "Belajar React" },
15   { id: 2, text: "Mengerjakan Praktikum" },
16   { id: 3, text: "Review Materi" },
17 ]);
18
19 // State untuk menyimpan teks yang sedang diketik di input
20 const [inputValue, setInputValue] = useState("");
21
22 // Fungsi menambahkan aktivitas baru ke dalam list
23 // Dipanggil saat user klik tombol "Tambah" atau tekan Enter
24 const handleAdd = () => {
25   // trim() menghapus spasi di awal/akhir; cegah aktivitas kosong ditambahkan
26   if (inputValue.trim() === "") return;
27
28   // Spread operator menyalin semua item lama, lalu tambah item baru
29   // Ini cara React-friendly untuk update array
```

```

9  function App() {
24  const handleAdd = () => {
30      const newActivity = {
31          id: Date.now(), // gunakan timestamp sebagai ID unik
32          text: inputValue.trim(),
33      };
34      setActivities([...activities, newActivity]);
35
36      // Reset input menjadi kosong setelah menambahkan
37      setInputValue("");
38  };
39
40  // Fungsi menghapus aktivitas berdasarkan id-nya
41  // Dikirim ke child component melalui props
42  const handleDelete = (id) => {
43      // filter() membuat array baru tanpa item yang id-nya cocok
44      setActivities(activities.filter((activity) => activity.id !== id));
45  };
46
47  // Menangani event tekan tombol keyboard di input
48  const handleKeyDown = (e) => {
49      if (e.key === "Enter") handleAdd();
50  };
51
52  return (
53      <div className="app-container">
54          <div className="card">
55              <div className="app-header">
56
57                  <div>
58                      <h1 className="app-title">Daftar Aktivitas</h1>
59                      <p className="app-subtitle">Mahasiswa Tracker</p>
60                  </div>
61              </div>
62
63              <div className="input-row">
64                  <input
65                      className="input-field"
66                      type="text"
67                      placeholder="Tambahkan aktivitas baru..."
68                      value={inputValue}
69                      // Setiap karakter diketik, state inputValue diperbarui
70                      onChange={(e) => setInputValue(e.target.value)}
71                      onKeyDown={handleKeyDown}
72                  />
73                  <button className="btn-add" onClick={handleAdd}>
74                      Tambah
75                  </button>
76              </div>
77
78              {activities.length > 0 && (
79                  <p className="activity-count">
80                      {activities.length} aktivitas tersimpan
81                  </p>
82              )}
83          </div>
84
85          {/* Child component ActivityList menerima data melalui props */}
86          {/* 'activities' dan 'onDelete' dikirim sebagai props ke ActivityList */}
87          <ActivityList activities={activities} onDelete={handleDelete} />
88      </div>
89  );
90  }
91
92  export default App;

```

Gambar C.1.3 Code App.jsx

```

src > components > ActivityList.jsx > ActivityList > activities.map() callback
1 // Child Component yang menerima data dari App (parent)
2 import ActivityItem from "../ActivityItem";
3 // Destructuring props langsung di parameter fungsi
4 // 'activities' = array data dari parent (App.jsx), 'onDelete' = fungsi hapus yang didefinisikan di parent
5 function ActivityList({ activities, onDelete }) {
6 // Jika array activities kosong, tampilkan pesan "Belum ada aktivitas"
7 if (activities.length === 0) {
8   return (
9     <div className="empty-state">
10       <span className="empty-icon">🕒</span>
11       <p>Belum ada aktivitas</p>
12       <span className="empty-hint">Tambahkan aktivitas di atas</span>
13     </div>
14   );
15 }
16 // Jika tidak kosong, render daftar menggunakan map()
17 return (
18   <ul className="activity-list">
19     {/* map() mengubah setiap item array menjadi elemen JSX, 'key' wajib ada agar React bisa melacak perubahan item */}
20     {activities.map((activity) => (
21       <ActivityItem
22         key={activity.id}
23         activity={activity} // Teruskan fungsi onDelete ke child berikutnya (ActivityItem)
24         onDelete={onDelete}
25       </ActivityItem>
26     ))}
27   </ul>
28 );
29 }
30 export default ActivityList;

```

Gambar C.1.4 Code ActivityList.jsx dalam folder components

```

1 // Child Component yang merepresentasikan satu item aktivitas
2 // Component ini menerima data dari ActivityList (parent-nya) via props
3
4 // Props yang diterima:
5 // - activity: object { id, text } data satu aktivitas
6 // - onDelete: fungsi untuk menghapus item (berasal dari App.jsx)
7 function ActivityItem({ activity, onDelete }) {
8   return (
9     <li className="activity-item">
10       <div className="item-left">
11         {/* Bullet dekoratif */}
12         <span className="item-dot" />
13         <span className="item-text">{activity.text}</span>
14       </div>
15
16       {/* Tombol hapus: memanggil onDelete dengan id item ini */}
17       {/* Arrow function dipakai agar onDelete tidak langsung dipanggil saat render */}
18       <button
19         className="btn-delete"
20         onClick={() => onDelete(activity.id)}
21         title="Hapus aktivitas"
22       >
23         ✕
24       </button>
25     </li>
26   );
27 }
28
29 export default ActivityItem;

```

Gambar C.1.5 Code ActivityItem.jsx dalam folder components

```

mc> # Appoin > % card
1 @import url('https://fonts.googleapis.com/css2?family=PlusJakartaSans:wght@400;500;600;700&display=swap');
2
3
4 *, *:before, *:after {
5   box-sizing: border-box;
6   margin: 0;
7   padding: 0;
8 }
9
10 :root {
11   --bg: #f0f2f5;
12   --card: #ffffff;
13   --accent: #2196f3;
14   --accent-hover: #00bcd4;
15   --danger: #f44336;
16   --danger-hover: #c2185b;
17   --text-primary: #212121;
18   --text-secondary: #757575;
19   --text-muted: #bdbdbd;
20   --border: #bdbdbd;
21   --shadow: 0 4px 24px #000(0, 0, 0, 0.07);
22   --radius: 14px;
23   --radius-sm: 8px;
24 }
25
26 body {
27   font-family: 'Plus Jakarta Sans', sans-serif;
28   background-color: var(--bg);
29   min-height: 100vh;
30   display: flex;

```

```

31   align-items: center;
32   justify-content: center;
33   padding: 24px;
34   color: var(--text-primary);
35 }
36
37 /* Layout utama */
38 .app-container {
39   width: 100%;
40   max-width: 480px;
41 }
42
43 /* Card */
44 .card {
45   background: var(--card);
46   border-radius: var(--radius);
47   box-shadow: var(--shadow);
48   padding: 12px 24px;
49   display: flex;
50   flex-direction: column;
51   gap: 20px;
52 }
53
54 /* Header */
55 .app-header {
56   display: flex;
57   align-items: center;
58   gap: 14px;

```

```

mc> # Appoin > % card
54 .app-header {
55   padding-bottom: 4px;
56   border-bottom: 1px solid var(--border);
57   padding-bottom: 20px;
58 }
59
60 .app-icon {
61   font-size: 2rem;
62   line-height: 1;
63 }
64
65 .app-title {
66   font-size: 1.4rem;
67   font-weight: 700;
68   color: var(--text-primary);
69   line-height: 1.2;
70 }
71
72 .app-subtitle {
73   font-size: 0.78rem;
74   color: var(--text-muted);
75   font-weight: 500;
76   margin-top: 2px;
77   text-transform: uppercase;
78   letter-spacing: 0.05em;
79 }
80
81 /* Input row */
82 .input-row {

```

```

83   .input-row {
84     display: flex;
85     gap: 10px;
86   }
87
88   .input-field {
89     flex: 1;
90     padding: 11px 14px;
91     border: 1.5px solid var(--border);
92     border-radius: var(--radius-sm);
93     font-family: inherit;
94     font-size: 0.9rem;
95     color: var(--text-primary);
96     background: #f5f5f5;
97     transition: border-color 0.2s;
98     outline: none;
99
100     .input-field:placeholder {
101       color: var(--text-muted);
102     }
103
104     .input-field:focus {
105       border-color: var(--accent);
106       background: #fff;
107     }
108
109     .btn-add {
110       padding: 11px 14px;

```

```

mc> # Appoin > % card
112 .btn-add {
113   background: var(--accent);
114   color: #fff;
115   border: none;
116   border-radius: var(--radius-sm);
117   font-family: inherit;
118   font-size: 0.88rem;
119   font-weight: 600;
120   cursor: pointer;
121   white-space: nowrap;
122   transition: background 0.2s, transform 0.1s;
123 }
124
125 .btn-add:hover {
126   background: var(--accent-hover);
127 }
128
129 .btn-add:active {
130   transform: scale(0.97);
131 }
132
133
134 /* Jumlah aktivitas */
135 .activity-count {
136   font-size: 0.78rem;
137   color: var(--text-muted);
138   font-weight: 500;
139   text-align: right;
140   margin-top: -8px;
141 }

```

```

142
143 /* Daftar aktivitas */
144 .activity-list {
145   list-style: none;
146   display: flex;
147   flex-direction: column;
148   gap: 8px;
149 }
150
151 /* Satu item aktivitas */
152 .activity-item {
153   display: flex;
154   align-items: center;
155   justify-content: space-between;
156   padding: 12px 14px;
157   border-radius: var(--radius-sm);
158   border: 1.5px solid var(--border);
159   background: #f5f5f5;
160   transition: border-color 0.2s, background 0.2s;
161   animation: slideIn 0.2s ease;
162 }
163
164 .activity-item:hover {
165   border-color: #888;
166   background: #fff;
167 }
168
169 @keyframes slideIn {
170   from { opacity: 0; transform: translate(-6px); }
171   to { opacity: 1; transform: translate(0); }
172 }

```

```

mc> # Appoin > % card
174 .item-left {
175   display: flex;
176   align-items: center;
177   gap: 10px;
178   flex: 1;
179   min-width: 0;
180 }
181
182 .item-dot {
183   width: 7px;
184   height: 7px;
185   border-radius: 50%;
186   background: var(--accent);
187   flex-shrink: 0;
188 }
189
190 .item-text {
191   font-size: 0.9rem;
192   color: var(--text-primary);
193   font-weight: 500;
194   white-space: nowrap;
195   overflow: hidden;
196   text-overflow: ellipsis;
197 }
198
199 /* Tombol hapus */
200 .btn-delete {
201   background: none;
202   border: none;

```

```

200 .btn-delete {
201   color: var(--text-muted);
202   font-size: 0.85rem;
203   cursor: pointer;
204   padding: 4px 6px;
205   border-radius: 6px;
206   transition: color 0.15s, background 0.15s;
207   line-height: 1;
208   flex-shrink: 0;
209   margin-left: 8px;
210 }
211
212 .btn-delete:hover {
213   color: var(--danger);
214   background: #f44336;
215 }
216
217
218 /* State kosong */
219 .empty-state {
220   display: flex;
221   flex-direction: column;
222   align-items: center;
223   gap: 8px;
224   padding: 32px 14px;
225   text-align: center;
226   color: var(--text-secondary);
227   border: 1.5px dashed var(--border);
228   border-radius: var(--radius-sm);
229 }
230

```

```

232 .empty-icon {
233   font-size: 2rem;
234   opacity: 0.5;
235 }
236
237 .empty-state p {
238   font-size: 0.95rem;
239   font-weight: 600;
240   color: var(--text-secondary);
241 }
242
243 .empty-hint {
244   font-size: 0.78rem;
245   color: var(--text-muted);
246 }

```

Gambar C.1.6 Code App.css

D. HASIL DAN DOKUMENTASI PROGRAM

Daftar Aktivitas

MAHASISWA TRACKER

Tambahkan aktivitas baru...

Tambah

4 aktivitas tersimpan

- Belajar React ×
- Mengerjakan Praktikum ×
- Review Materi ×
- Laporan Praktikum ×

Daftar Aktivitas

MAHASISWA TRACKER

Tugas Besar UX

Tambah

4 aktivitas tersimpan

- Belajar React ×
- Mengerjakan Praktikum ×
- Review Materi ×
- Laporan Praktikum ×

Daftar Aktivitas

MAHASISWA TRACKER

Tambahkan aktivitas baru...

Tambah



Belum ada aktivitas

Tambahkan aktivitas di atas

E. ANALISIS DAN PEMABAHASAN

Pada praktikum ini dilakukan pengenalan React JS sebagai library JavaScript untuk membangun antarmuka pengguna berbasis component. Praktikum dimulai dari proses setup environment menggunakan Node.js, npm, dan Vite hingga aplikasi berhasil dijalankan pada localhost. Dari proses tersebut dapat dipahami bahwa React memiliki struktur project yang lebih terorganisir dan mempermudah pengembangan aplikasi web.

Component digunakan untuk membagi tampilan aplikasi menjadi bagian kecil yang dapat digunakan kembali, sehingga kode menjadi lebih rapi dan efisien. JSX mempermudah penulisan tampilan karena sintaksnya mirip HTML tetapi tetap berada di dalam JavaScript.

Pada bagian props dan state, praktikum menunjukkan bagaimana data dapat dikirim dari parent component ke child component menggunakan props, sedangkan state digunakan untuk menyimpan data yang dapat berubah. Perubahan state akan membuat tampilan aplikasi diperbarui secara otomatis tanpa perlu memanipulasi DOM secara manual.

Selain itu, digunakan juga rendering list dan conditional rendering untuk menampilkan data secara dinamis berdasarkan kondisi tertentu. Pada tugas akhir praktikum, seluruh konsep tersebut diterapkan dalam aplikasi daftar aktivitas mahasiswa yang dapat menambah, menampilkan, dan menghapus data aktivitas menggunakan state dan component React.

F. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa React JS merupakan library JavaScript yang mempermudah pembuatan antarmuka pengguna melalui konsep component-based architecture. React menggunakan JSX, props, state, rendering list, dan conditional rendering untuk membangun aplikasi yang lebih dinamis dan interaktif. Penggunaan component membuat kode menjadi lebih terstruktur dan reusable, sedangkan state dan props membantu pengelolaan data antar component dengan lebih teratur. Melalui praktikum ini, mahasiswa dapat memahami dasar pengembangan aplikasi React sederhana mulai dari setup project hingga implementasi fitur interaktif pada aplikasi frontend.

G. LAMPIRAN

<https://github.com/AriqAhmadNayaka/WebPro4904.git>